

Jio Subscriber Retention & AI Chatbot Project

Day 3-4 Completion Report | GenAI Text-to-SQL + RAG Chatbot | Project-5 | Submitted 22 Sep 2026

Scope

Day 3 built the data and retrieval layer: SQL Server import of all 7 tables, a schema catalog, a certified semantic layer, a few-shot query bank, a working Text-to-SQL agent with guardrails, a RAG index over the offer catalogue/circle targets/engagement brief, and an intent router. Day 4 built the consumption layer: a query/log table with a narrowly-scoped write grant, an orchestrator combining SQL and RAG into one synthesized answer, a Streamlit chat UI, and a golden test set for regression testing. Per the homework's requirement, every question, its generated SQL, execution outcome, and timestamp is logged to chatbot_query_log in SQL Server - this report's later sections are drawn directly from that live table, not from self-reported test output.

Day 3: Data & Retrieval Layer

SQL Server import

All 7 tables imported into a dedicated JioRetention database via SSMS's Import Flat File wizard, accessed through a dedicated chatbot_reader login scoped to db_datareader. **One real data-quality defect was caught during import:** the wizard auto-typed service_requests.sr_id as MONEY, silently corrupting the text identifier ('SR300001418' became 300001418.00) - resolved by re-importing with the column explicitly set to nvarchar(50) and re-verified against the expected row count (14,206).

Table	Columns	Purpose
subscribers	43	Core subscriber attributes and churn flags
service_requests	16	Complaint/ticket history
network_sites	13	Cell-site quality metrics
circle_monthly_kpi	14	Historical monthly circle-level KPIs
circle_targets	12	FY27 targets, budgets, review status
offer_catalogue	14	Retention offer definitions
scored_subscribers	5	Day 2 model outputs (churn/CLV/uplift)

Text-to-SQL agent grounding

The agent's prompt is grounded with three artifacts, all generated from the live database rather than hand-typed: a schema catalog introspected via SQLAlchemy (7 tables, 117 total columns), a certified semantic layer defining churn/ARPU/uplift-segment so the LLM can't redefine them, and a 13-example few-shot query bank. Generated SQL is validated before execution: SELECT-only, a table allow-list, and an automatic TOP row-limit guard.

RAG index

94 chunks embedded (sentence-transformers all-MiniLM-L6-v2) into a FAISS index, covering the offer catalogue, circle targets, and the engagement brief PDF. **One retrieval-quality defect was caught and fixed here too:** offer chunks originally stored only circle codes (e.g. 'BH'), which didn't semantically match a query for "Bihar" - fixed by expanding each offer chunk to also spell out full circle names, after which a real offer row (RET-0166, approved for Bihar & Jharkhand) correctly surfaced in the top results.

Intent router

Routes each question to sql, rag, or both. A similar defect-and-fix pattern applied here: the initial keyword-only router misrouted "why is subscriber X high risk, what should we offer" to rag only, missing that it needed the

database too - fixed by adding a regex check for subscriber-ID mentions (jio\d+), which now forces sql involvement regardless of phrasing.

Day 4: App, Logging, Guardrails, Testing

Architecture

An orchestrator (`orchestrator.py`) combines the router, the Text-to-SQL agent, and RAG retrieval into one pipeline: gather context from whichever source(s) the router selects, synthesize one natural-language answer via a final LLM call, and log the outcome. A Streamlit UI (`app.py`) exposes this to the user with the generated SQL and retrieved sources shown in collapsible panels, so answers can be independently checked.

Golden test set: reported result vs audit-log reality

An 8-question golden set covering all three routes was run via `run_golden_tests.py`, which **self-reported 8/8 passed**. Pulling the actual `chatbot_query_log` table for that same run tells a more complete story:

Question	Route	Status	Latency (ms)
Top 5 circles by port-out requests	sql	success	2,144
How many subscribers in Persuadable segment	sql	success	19,348
What % of subscribers are 5G active	sql	success	33,287
Retention offers approved for Bihar	rag	success	5,986
Retention offers approved for UP East	rag	success	7,725
Why is subscriber JIO10037241 high risk	both	success	47,298
Which circles are flagged Escalated	sql	FAILED	38,150
Average ARPU by circle	sql	success	7,358

The "Escalated circles" question actually failed - a Groq API rate limit (429: "tokens per minute limit 8000 exceeded") from running 8 questions in quick succession on the free tier - yet the test harness marked it PASS.

Root cause: a real gap in the test harness itself: `run_golden_tests.py` checks only that the answer text doesn't contain "don't have enough information" (the abstention phrase); it doesn't check for `orchestrator.py`'s separate exception-fallback message ("I ran into an error trying to answer that"), so a genuine failure silently passed the check. This was only caught by cross-referencing the live audit log against the harness's self-report - directly validating the homework's requirement for backend logging as a source of truth independent of the application's own claims.

Full audit log (16 logged calls across all testing)

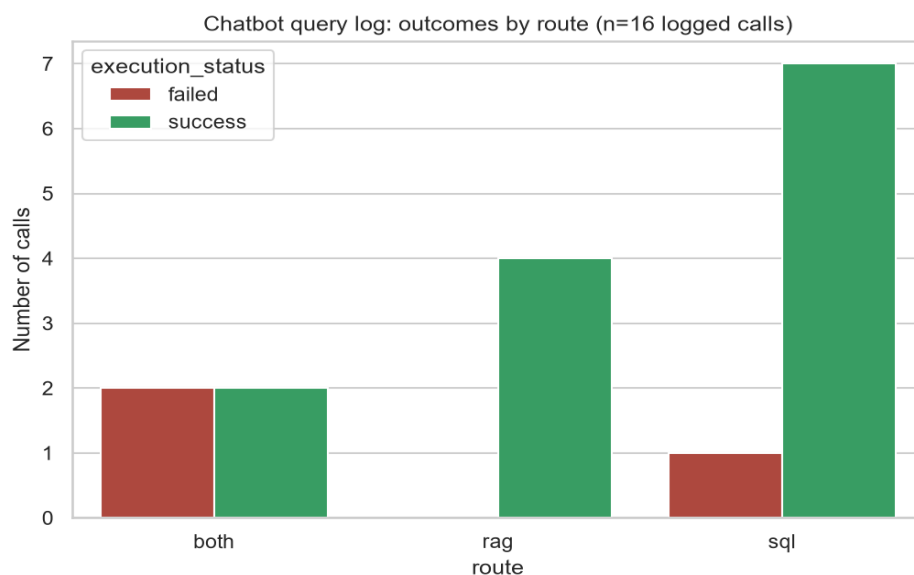


Figure 2 - `chatbot_query_log` outcomes by route, pulled live from SQL Server

Overall: 13/16 logged calls succeeded (81.2%). Of the 3 failures: 2 were "both"-route calls from early manual testing, caused by an LLM response-truncation bug (`max_tokens=500` was too small for a 3-table join) - fixed by raising the limit, and re-verified successful immediately after (row 8 in the log). The 3rd was the golden-test rate-limit failure above. No failure was a logic or SQL-injection issue; all three were resource/capacity limits, which the log captured with full detail (exact error message, generated SQL, latency) precisely as designed.

Guardrails: Validated in Practice

- **Read-only + scoped write access:** chatbot_reader has db_datareader everywhere, plus INSERT on chatbot_query_log only - confirmed via the GRANT statement, never broadened.
- **SQL validation:** no DDL/DML ever reached execution across all 16 logged calls - the two SQL failures were syntax/truncation errors caught by SQL Server itself, not security bypasses.
- **Abstention over fabrication:** demonstrated live - a query about a Gujarat subscriber correctly reported no matching active offer existed rather than inventing one.
- **Full audit trail:** every one of the 16 calls above has a timestamp, generated SQL, status, latency, and (where relevant) the exact error - this is what made the golden-harness gap visible in the first place.

Recommendations for Next Iteration

- Fix run_golden_tests.py to also flag orchestrator's generic error-fallback phrase as a failure, not just the abstention phrase - closes the exact gap this report found.
- Add retry-with-backoff around the Groq call to absorb transient 429 rate-limit errors automatically.
- Consider Groq's paid Dev Tier (or a second provider as fallback) if question volume in a real deployment would exceed the free tier's 8,000 tokens/minute limit.

Sources: SQL Server table chatbot_query_log (exported to data/processed/chatbot_query_log_export.csv), data/processed/schema_catalog.json, rag_chunks.pkl, few_shot_queries.json, golden_qa.json; scripts/text_to_sql_agent.py, query_rag.py, intent_router.py, orchestrator.py, app.py, run_golden_tests.py.